CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem

How a model works — and how we should use it

Bowen Liu Draft v0.1 45 min

What the model sees: ten integers

EN · ONE SENTENCE

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 characters → **10** tokens → 10 integers · _ marks a leading space

CHINESE · FOUNDATIONS OF CELESTIAL MECHANICS

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 characters → **10** tokens → 10 integers · two-character words often map to one token

“This book systematically introduces celestial mechanics.”

01 Where do these integers come from?

02 How does it predict the next one?

03 How do we put it to work?

Tokenization is table lookup

Three-body problem is a classic problem in celestial mechanics.

Three	-body	problem	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
25843	19146	4792	382	261	13686	4792	306	134437	52347	13

_ marks a leading space · Celestial → Cel + estial

≈5
characters / English word
including the space, typical

≈4
characters / token
English rule of thumb

≈1.3
tokens / English word
≈ 5 ÷ 4

50k–200k
typical vocabulary size
50k – 200k

WHY Characters: little information, 4–5× longer → Words: vocabulary explodes (millions) → **Subwords: the compromise**

GPT-2 50k · GPT-4 100k · Llama 3 128k · GPT-4o 200k

> Subwords are an engineering compromise between **sequence length** and **vocabulary size**.

English: 1 token ≈ 4 characters ≈ 0.75 words

The vocabulary: a fixed lookup table

Foundations of Celestial Mechanics · row number = ID · sorted by ID

ID	TOKEN	
290	_the	
2392	_book	
2500	This	
18864	_fundamental	
23753	_concepts	
52347	_mechanics	
134437	_celestial	← query
199999	<|endoftext|>	special

8 rows shown · 200,019 rows in total

Vocabulary size = number of rows

200,019

o200k · GPT-4o generation

- **0**
- **50k** GPT-2
- **100k** GPT-4 gen
- **128k** Llama 3
- **200k** GPT-4o gen

200,019 × 4,096 ≈ **820M parameters**

> An ID is a **row number**; changing models means changing the table.

Tokens: the model's unit and the billing unit



Last month · ≈110k tokens per request · ≈¥0.01 each

≈ 0.3 token
per English character

≈ 0.6 token
per Chinese character

Rule of thumb: a word, a number or a symbol ≈ 1 token; tokenizers differ, so trust the usage the API returns.

> Token is both the smallest unit of the model and the smallest unit of the bill.

Where the money goes on a single call

价格[2]	百万tokens输入（缓存命中）	空闲时段	0.02元
		高峰时段	0.04元
	百万tokens输入（缓存未命中）	空闲时段	1元
		高峰时段	2元
	百万tokens输出	空闲时段	4元
		高峰时段	8元

deepseek-flash · 1M context · 384K output · CNY per million tokens

1M context ≈ the whole Three-Body trilogy

One call · off-peak

Input 100k + output 2k

¥0.01 cache hit

¥0.11 cache miss

> The same context, cached or not, differs by an **order of magnitude**.

Text becomes an ID, then a vector

PIECE

`problem`

A unit after tokenization

→

ID

`4792`

Row 4792 of the vocabulary

→

VECTOR

`0.12 −0.44 0.08 …`

4,096 dimensions · learned

> The ID is an **address**; the vector is the **content**.

This step is called embedding; no dimension has a name, yet distance carries meaning.

What the model does: input → probability distribution

Input tokens

From | the | next | token | to | the

Each token is already a vector

Model

Transformer

N layers · self-attention

GPT · Llama · Qwen are all variants

softmax(z)



$$z = f_\theta(x_{1:t}) \in \mathbb{R}^{|V|} \qquad p_\theta(y \mid x_{1:t}) = \frac{\exp(z_y)}{\sum_{v \in V} \exp(z_v)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

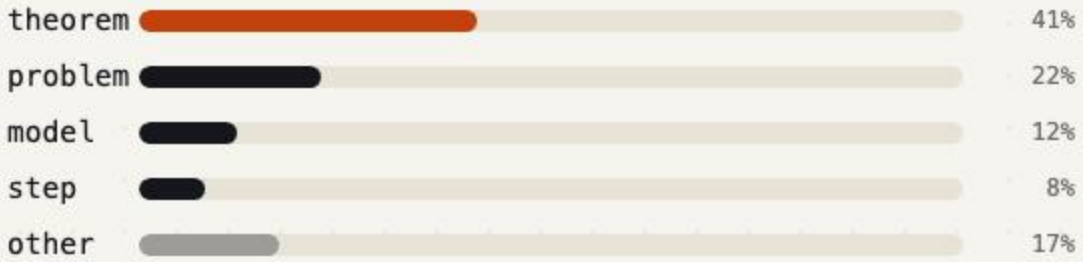
> A scorer: it gives every token in the vocabulary a **score**, then turns scores into **probabilities**.

Sampling: draw one token from the distribution

Probability distribution · prefix

"One important …"

$$y_{t+1} \sim p_\theta(\cdot \mid x_{1:t})$$



GREEDY

Take the max → **theorem**

SAMPLING

Draw by probability → this time maybe **problem**

KNOBS

temperature · **top-p** control randomness

Input → Transformer → Probability distribution → Sampling → Append to input ↻

> The same prompt can give two different results — that is **sampling**, a **dice roll** by design.

Sampling: throw a random number onto the CDF

token	p	F(y)
theorem	0.41	0.41
problem	**0.22**	**0.63**
model	0.12	0.75
step	0.08	0.83
other	0.17	1.00



u = 0.57 ∈ [0.41, 0.63) → pick “problem”

TEMPERATURE

$$p_i^{(T)} = \frac{p_i^{1/T}}{\sum_j p_j^{1/T}}$$

Small T → sharper; large T → flatter

TOP-P

$$\min |S| \text{ s.t. } \sum_{i \in S} p_i \geq 0.9$$

Trim the long tail, then renormalize

> Temperature reshapes, top-p trims the tail, **u** decides where it lands.

The model is stateless

CALL A

```
system    You are a celestial mechanics assistant
user      What is the Hill region?
assistant The Hill region is …
user      Why does it matter?
assistant Because it defines …
```

CALL B · A FRESH CALL

```
system    You are a celestial mechanics assistant
user      Why does it matter?
assistant ("it" has no referent here)
```

> The model is stateless; a "conversation" is the client **resending** the history.

Parameters stay fixed at inference time; state lives only in the context.

From next token to problem solving

PRETRAIN

Knowledge

Reads the corpus, learns language and the world

SFT

Behavior

Sees demonstrations, learns to follow instructions

RL

Policy

Solves problems, checks answers, learns to search and correct

> The model stays the same; the **training signal** changes.

Pretrain: read the internet once

$$L(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

Given the preceding text, predict the next token

DATA

Trillions of tokens

Web pages, code, papers, books

COMPUTE

1k–10k GPUs

Months of training

OUTPUT

base model

Completes text; SFT teaches it to answer

> It learns the world's **statistical structure**; tasks come later.

The first 20 tokens, 19 samples

Foundations of Celestial Mechanics · blurb

This book systematically introduces the fundamental concepts and theorems of celestial mechanics, together with a selection of representative research results. It has five chapters: the two-body problem; the restricted three-body problem; the perturbed two-body problem; resonance phenomena in celestial motion; and order and chaos in conservative systems.

First 20 tokens

`This` `_book` `_systematically` `_introduces` `_the` `_fundamental` `_concepts` `_and` `_the` `ore` `ms` `_of` `_celestial` `_mechanics` `,` `_together` `_with` `_a` `_selection` `_of`

INPUT (PREFIX)	TARGET
This	**book**
This book	**systematically**
This book systematically	**introduces**
This book systematically introduces	**the**
This book systematically introduces the	**fundamental**
This book … a selection	**of**

19

training samples

Every position predicts the next token

$$L = -[\ \log p(\text{book} \mid \text{This}) + \log p(\text{systematically} \mid \text{This book}) + \ldots + \log p(\text{of} \mid \text{first 19 tokens})\ ]$$

> Training runs in parallel; inference runs one token at a time.

What this loss is

Negative log-likelihood · cross-entropy

P(CORRECT TOKEN)	−LOG P
1.00	0
0.50	0.69
0.10	2.30
0.01	4.61

Units: nats · ÷ ln 2 = bits

Perplexity PPL = exp(mean −log p)

∂L/∂z = p − onehot(y)

SINGLE POSITION

−log p = surprise, and the code length of this token

WHOLE SENTENCE

Sum = total code length; average, then exp, gives perplexity

MINIMIZE

= maximum likelihood = optimal compression

> This loss carries meaning: it is the **code length** the model assigns to the data.

Compression is intelligence

To predict well,
you must **compress the world** into the parameters.

PREDICT
Explain every token
Skip no detail

COMPRESS
Squeeze out redundancy
Keep the reusable structure

STRUCTURE
Grammar · causality · physics
Transfer to unseen problems

Compression explains where knowledge comes from; search and correction come from RL, later.

> This is the sentence I have believed most in recent years.

SFT: learn the job from worked answers

supervised fine-tuning · instruction tuning

system	You are the teaching assistant for Foundations of Celestial Mechanics.	input
user	What is this book about?	input
assistant	It systematically introduces the fundamental concepts and theorems of celestial mechanics.	target

Orange = the only tokens that contribute to the loss.

SFT $-\Sigma_{t \in \text{answer}} \log p(y_t \mid \text{prompt}, y_{<t})$

DATA

(instruction, answer) pairs

OBJECTIVE

Still next-token prediction; the condition is system + user + partial answer

BOUNDARY

Behavior cloning: it imitates demonstrations. Knowledge comes from pretraining; hard problems can still fail.

> Same loss, different **summation range**.

One message, three roles

SYSTEM
You are the teaching assistant for Foundations of Celestial Mechanics. Answer only from the book.

USER
What is Chapter 4 about?

ASSISTANT
Resonance phenomena in celestial motion.

SYSTEM
Identity, rules, boundaries; written by the developer

USER
Task and material; written by the user

ASSISTANT
The model's answer; the only part that carries loss

`system / user = condition · assistant = supervision target (loss on these tokens only)`

> system sets the rules, user sets the task, assistant is the learning target.

How it actually predicts

GENERATED SO FAR	CANDIDATES (PROBABILITY)	SAMPLED
(empty)	This .62 · The .11 · It .08	**This**
This	book .48 · volume .12 · text .09	**book**
This book	systematically .71 · provides .06 · introduces .05	**systematically**
… systematically	introduces .66 · covers .12 · presents .08	**introduces**
… introduces	the .58 · a .21 · fundamental .07	**the**
… the	fundamental .44 · basic .18 · core .09	**fundamental**

Probabilities are illustrative, to show the mechanism · real outputs follow the model

01 SCORE

Given every token generated so far

02 SAMPLE

Draw one by probability

03 APPEND

Add it to the input and repeat

> One step at a time: append the result, then take the next step.

What reinforcement learning is

Trial and error in an environment, using a reward signal to adjust the policy so long-run return is maximal.



NO ANSWER KEY

Only reward

It scores the outcome and leaves the choice to the agent

TRIAL AND ERROR

Action → feedback

Reinforce what works, revise what fails

GOAL

Cumulative reward

Long-run return over the immediate step

Objective · maximize expected discounted return

$$\max_{\pi} E\left[\sum_t \gamma^t r_t\right]$$

π policy · γ discount · r_t reward · E expectation

> What RL learns is a **policy**.

agent · environment · reward

RL's old battlefield: win games first

2013

Atari · DQN

Learn to play from pixels

2016

AlphaGo

Self-play + tree search

2018

Dota 2 · OpenAI Five

Long horizons, team play

2019

StarCraft II · AlphaStar

Imperfect information, huge action space

- has a simulator
- has a reward
- unlimited retries

DeepMind: Atari · AlphaGo · AlphaStar (RL) | AlphaFold → 2024 Nobel Prize in Chemistry (Hassabis & Jumper)

> **AlphaFold is supervised learning:** labels are known structures; no reward, no trial and error.

> Math and code bring their own **simulator**.

RL's new battlefield: solve, then check

$$\max_{\theta} E_{y \sim p_\theta}\left[r\left(y\right)\right] - \beta \mathrm{KL}\left(p_\theta \middle\| p_{\mathrm{ref}}\right)$$

Reward comes from programs: answer matching, unit tests, Lean compilation

Q: What is Chapter 4 about?

Perturbed two-body problem	0
Resonance phenomena in celestial motion	**1**
Order and chaos	0

Sample 3 → verifier scores → update policy

How many r's are in "strawberry"? s t r a w b e r r y GPT-4o often says 2 → **o1 / R1 says 3** Split the letters, then count

2024.09 o1 → 2025.01 R1 · the DeepSeek moment

> Same text, new objective: the reward arrives only when the answer checks out.

Common RL algorithms

PPO

Policy gradient + critic

The classic choice for RLHF

GRPO

Group-relative advantage

Drops the critic; common in math and code

RLOO

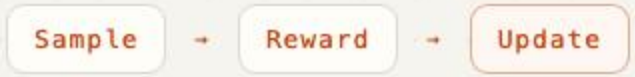
Leave-one-out baseline

Lightweight, lower variance

DPO

Learn preferences directly

Skips the RL loop



PPO / GRPO / RLOO are RL; DPO is preference optimization, usually placed in the RLHF family.

> Different algorithms, same loop: sample → score → update.

Math and code: a natural advantage

	GENERAL TASKS	MATH / CODE
REWARD	Human rating / reward model	**Program verdict**
COST	Expensive, slow, noisy	**Near zero**
SUPPLY	Limited	**Unlimited generation**
FEEDBACK	A single score	**The exact error location**

01 DECOMPOSE

Functions, modules → subgoals

02 VERIFY

Tests → self-check

03 DEBUG

Error → locate → fix

04 LONG-HORIZON CONSTRAINTS

Types, variables, control flow → state preserved

> Clearer feedback makes trial and error cheaper — and RL more effective.

From talking to working

TALKING · 2023

ChatGPT

Instruction tuning + RLHF
prompt / system

THINKING

CoT

Write the intermediate steps
context engineering · pruning

WORKING

Agent

tool calling · loop
memory

> Each step solves the problem the previous step left open.

2023: from completion to conversation

```
user       Explain the Hill region in one sentence.
assistant  The region of space a body can reach at a given energy.
```

INSTRUCTION TUNING

Learns the dialogue format: respond to the request

RLHF

Learns answers people prefer: helpful, harmless

SCOPE

Text only: no tools, no memory, no verification

> Conversation is a trained behavior.

Prompt: change the distribution with content

SYSTEM · ROLE
You are a celestial mechanics teaching assistant.

CONSTRAINTS
Write the formula first, then substitute, then give the number.

USER · TASK
A period of 2 years; the semi-major axis grows 4×. What is the new period?

FEW-SHOT · EXAMPLES
a × 9 → T × 27
a × 16 → T × 64



$$p_\theta(y \mid x) \rightarrow p_\theta(y \mid x, c, e)$$

p(16 | x) = 0.18
p(16 | x, c, e) = 0.71

> Adding content is how you change the **conditional probability**.

CoT: write the thinking down

Question

A period of 2 years; the semi-major axis grows 4×. What is the new period?

DIRECT ANSWER

8 years ×

STEP BY STEP

$T \propto a^{3/2}$; $4^{3/2} = 8$; $T = 2 \times 8 = 16$ years ✓

WHY IT WORKS

Every token is extra computation

COST

Slower, pricier, sometimes circular

ESSENCE

Spread implicit reasoning into explicit steps

> Writing it out is computing it out.

The inflation of the context window



COST

Billed per token; long prefills are slower

LATENCY

Prefill and attention costs rise

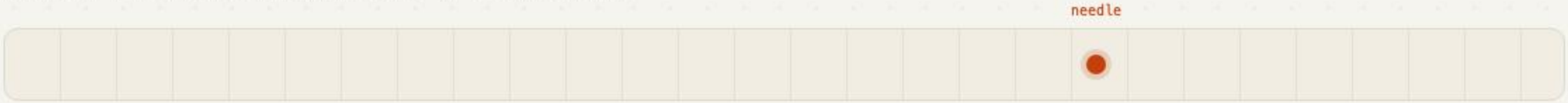
DILUTION

Middle content gets ignored; signal drowns in noise

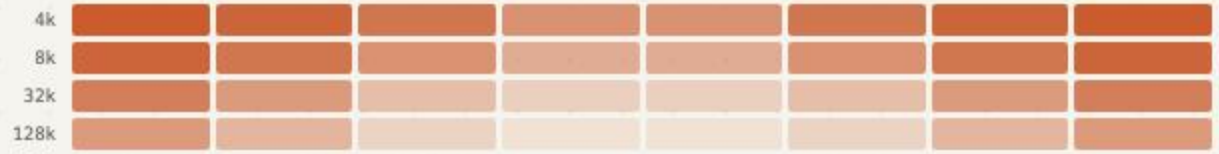
> A longer window raises the ceiling; relevance decides what lands.

Context is finite

Needle in a haystack: hide one sentence in a long document



Retrieval accuracy (illustrative)



Horizontal axis: needle position 0 → 100%

METHOD

Hide a sentence, vary length and position, ask what it was

RESULT

Harder as length grows; worst in the middle

IMPLICATION

A big window is a budget; retrieval still matters

> Long context is a budget; retrieval decides the recall.

Context engineering: the window is a workbench

What goes in the window

```
system: rules
history: recent turns
retrieval: relevant documents
tools: returned results
current: this question
```

TOO LONG

Retrieve; keep only what is relevant

TOO SCATTERED

Structure it; label the blocks

LOST IN THE MIDDLE

Put key facts at the start and end

OVERFLOW

Compress, summarize, sub-agents

> Relevance beats length.

CoT pruning: keep the thoughts that matter

A long chain of thought

Read problem → Name variables → Expand → ~~Repeat~~ → Case split → ~~Side branch~~ → Main line → Substitute → Check → Answer

PRUNE

Drop repeated and irrelevant branches

DISTILL

Teach a short CoT from a long one

SUMMARIZE

Compress a long chain into a short one

BUDGET

Cap thinking tokens; stop early

> Thinking has a budget too.

Tool calling: let a program do the arithmetic

```
Tool: run_python(code: str) → str

call   run_python("print(2 * 4**1.5)")
obs    16.0

// T = 2 × 4**1.5 = 16
```

01 DEFINE

Tool signature: name, arguments, return

02 CALL

The model emits a snippet

03 EXECUTE

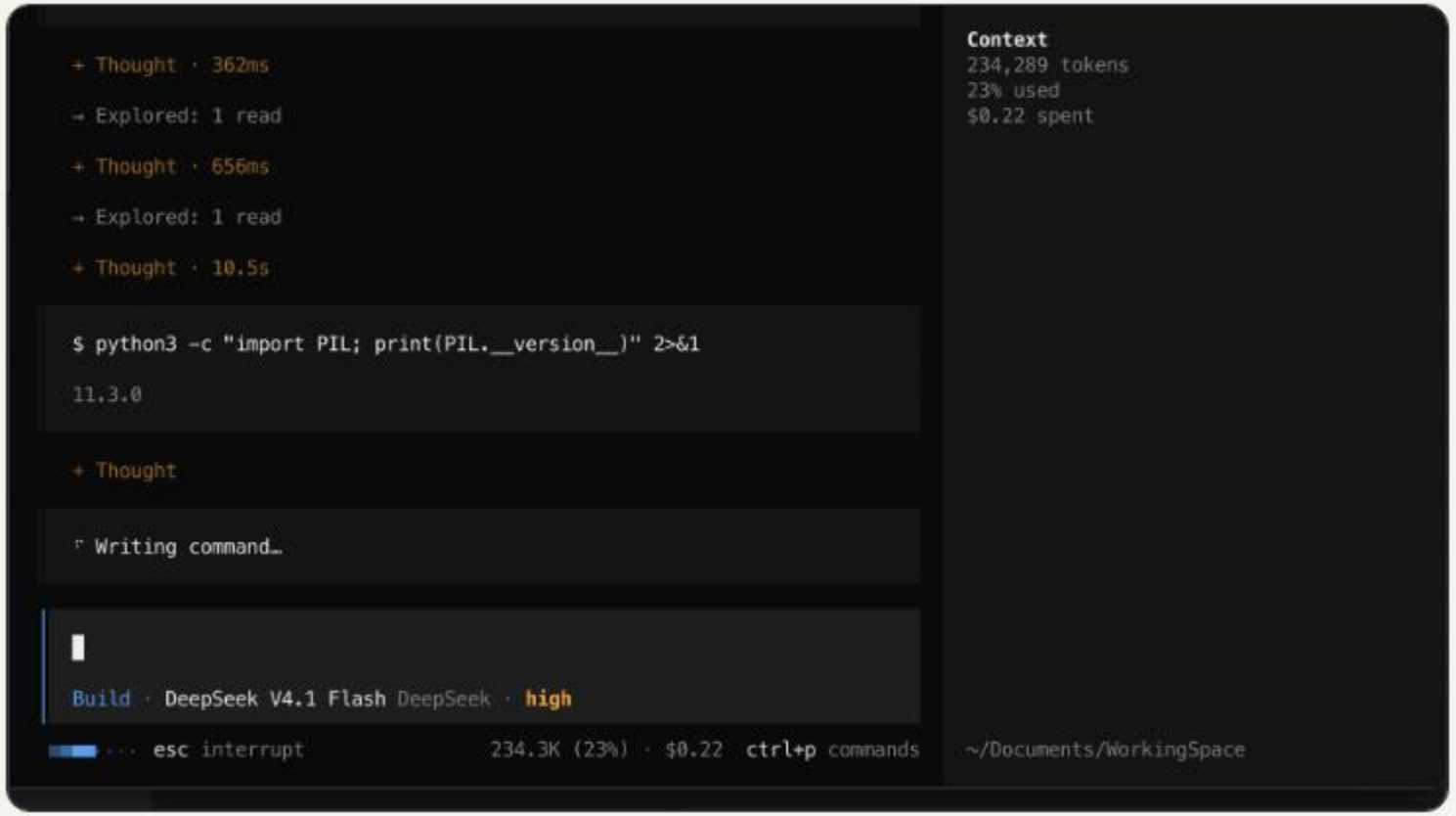
It really runs, in a sandbox

04 OBSERVE

The result returns to the context; continue

> The model proposes; the program computes.

Agent: the model inside a loop



01
Loop
Thought → tool call → observation, repeat

02
Context
234K tokens · 23% used · $0.22

03
Client
Model + tools + session + billing

04
Model
DeepSeek V4.1 Flash · high

> The model proposes; the client turns proposals into **actions**.

Memory: signal and noise

What is in the window

Signal
Task · constraints · decisions · verified facts

Noise
Chat · repetition · stale info · raw logs

Write
Store decisions and facts only

Retrieve
Fetch for the current task

Compress
Summarize, merge

Forget
Delete when stale

WORKING MEMORY

The current window; only what this step needs

EPISODIC MEMORY

Past summaries; decisions and open questions

SEMANTIC MEMORY

External store + retrieval; fetch on demand

PROCEDURAL MEMORY

system + tools; rules and capabilities

> Memory works by selection: store little, retrieve precisely.

Harness: catch the rare failure

> LLMs are probabilistic: a 0.1% failure rate × 1M requests = **1,000 failures**

System · rules

ROLE
Identity, rules, output format

LIMITS
Prompts guide; the runtime enforces

PRINCIPLE
Rules fixed in the prompt; permissions in the runtime

Harness · the runtime scaffold

VERIFY
Tests, compilation, Lean

RETRY
Rerun, roll back, change route

PERMISSIONS
Sandbox, allowlist, budget

OBSERVE
Logs, replay, evals

> The model handles probability; the harness handles certainty.

Put the model back in the system



Inner ring: one call · Middle ring: across calls · Outer ring: engineering boundary

01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> The model is the CPU; prompt, context, memory, loop and harness are the runtime.

Acceleration: from basic use to gold medals

2023 · BEFORE

Basic use

- Explain concepts, search the literature, write code
- Do simple calculations, often wrong
- Complex proofs still need people
- Role: assistant

→ 19 months

2025 · AFTER

Research level

- IMO 2025 gold level 35/42
- Lean proofs, machine-checkable
- OpenAI 2026: 372 problems, ≈3 h each (report)
- Role: candidate generator

2024.07 **28/42** silver → 2025.07 **35/42** gold level — 12 months, +7 points

> This is a staircase with **three steps**.

Timeline: three steps

2023–24

Tools: search + verification

2023.12
FunSearch
LLM + evaluator + evolutionary search; a new cap-set construction

2024.01
AlphaGeometry
Neuro-symbolic; IMO geometry 25/30

2024.07
AlphaProof
Lean + RL; IMO 2024 silver 28/42 (+AG2)

2024–25

Reasoning: RL + test-time compute

2024.09
OpenAI o1
RL teaches thinking; test-time compute

2025.01
DeepSeek-R1
RLVR; reasoning behavior emerges; open weights

2025.05
AlphaEvolve
Evolutionary coding agent; 4×4 complex matrix product in 48 multiplications

2025–26

Scale: from benchmarks to open problems

2025.07
IMO 2025
Gold level 35/42; natural language, no tools

2025
Formal gold (claimed)
Aristotle et al.; Lean-checkable

2026.05–10
OpenAI
Unit-distance counterexample; NS / Riemann-related (claimed); 372 problems (report)

> Every leap combines a **generator, a verifier and search**.

How they describe it

> Public material covers part of the method; the full training recipe stays private. Note the asymmetry: some results are easy to verify, hard to prove. Quotes below are **public statements**, excerpted or paraphrased.

DATE	SYSTEM / WORK	PUBLIC STATEMENT	READING
2023.12	**FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	search
2024.01	**AlphaGeometry**	"100 million synthetic theorems and proofs"	synthetic data
2024.07	**AlphaProof**	autoformalization → Lean; RL + test-time RL	formalization
2024.09	**o1**	"spend more time thinking through problems"	reasoning
2025.01	**R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.05	**AlphaEvolve**	"evolutionary coding agent"	evolve code
2025.07	**IMO 2025**	"natural language ... no tools"	end to end
2026.09–10	**OpenAI**	"N5 solution; 372 problems; 722 papers"	open problems

> The keywords change; the underlying move stays: **propose, verify, search again**.

The common pattern: generate → verify → search

GENERATE

LLM

Candidate proofs, programs, constructions

→

VERIFY

Lean / tests / numerics

Returns right or wrong, a score, a counterexample

→

SEARCH / LEARN

best-of-N · trees · evolution · RL

Turns the signal into the next policy

↺ Back to the generator: the next round is stronger

MANUAL ONLY

Review stays manual; scale stalls

ONE SHOT

One draw per attempt; luck decides

BOTH

Probability is filtered into correctness, then written back into the model

> The model supplies probability; **the verifier supplies the gradient**.

Some guesses: how the big results may have happened

> The following mechanism hypotheses come from public papers, blogs, and benchmarks, and are labeled as **guesses**.

01 DATA

Autoformalization + synthesis: informal problems → Lean; rejection sampling keeps the verifiable ones.

02 TRAINING

RLVR + long CoT: verifiable answers let rewards scale.

03 INFERENCE

Test-time search: best-of-N, tree search, evolution.

04 DISTILLATION

Write search back into the model: less search next time.

05 ENGINEERING

Harness: parallelism, caching, evals, replay.

06 THE BET

Verification: the harder your verifier, the faster your flywheel.

OpenAI: 372 problems, ≈3 hours each (report)

> Three hours per problem looks more like **parallel sampling + verification + search**.

What remains: from solving to doing mathematics

Already happening

CONTEST PROBLEMS
IMO gold level; natural language, timed

FORMAL PROOFS
Lean-checkable; machine verification

CONSTRUCTION SEARCH
Cap sets, matrix multiplication, scheduling

OPEN PROBLEMS
372 in the OpenAI report; humans still curate

Still open

CHOOSING A PROBLEM
Which question is worth asking

DEFINITIONS
New concepts, new structures

LONG HORIZON
A research plan spanning months

TASTE
Beauty, importance, growth

> Solving can be searched; **doing mathematics needs direction**.

The algorithm engineer's dilemma

It arrived about a year ago. Mathematics is next.

"If the model can write the code, what is my job?"

BEFORE AGENTS

Write the algorithm

- Implement, debug, optimize
- Own the code
- Value = code shipped

→ ~1 year

WITH AGENTS

Direct the search

- Frame the problem
- Design the harness
- Verify and own the system

> The job moves from producing answers to **directing the search**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.